

Universitat Politècnica de Catalunya
ETSETB

Aprenentatge no supervisat i detecció d'anomalies

Document de descripció dels problemes de l'exposició

Computació d'Altes Prestacions i Intel · ligència Artificial (CPIA)

Equip:
Martí Canut Sumarroca
Uriel Martínez Rosa
Arnau Montull Bainad
Matías Stein Niell

Grau en Enginyeria Física



27 d'octubre de 2026

Índex

1 Equip i repartiment dels problemes **2**

2 Fonaments comuns **2**

3 Problema d'Arnau Montull: detecció d'explosions nuclears amb DBSCAN **3**
3.1 Interès del problema . . . 3
3.2 Plantejament de la solució . . . 3
3.3 Dades . . . 4
3.4 Fonamentació teòrica . . . 5
3.5 Mètode: com es detecta . . . 5
3.6 Elecció dels paràmetres . . . 7
3.7 Validació de ε i estabilitat (`nukeval.py`) . . . 8
3.8 Resultats . . . 9
3.9 Què hi ha darrere de cada clúster . . . 10
3.10 Conclusions . . . 12
3.11 Com reproduir-ho . . . 12

4 Problema de Matías: agrupació de sismes i detecció d'anomalies **12**

5 Problema de Martí: expressió gènica i clustering jeràrquic (leucèmia ALL/AML) **13**
5.1 Plantejament del problema . . . 13
5.2 Fonament teòric . . . 14
5.3 Metodologia . . . 14
5.4 Resultats i anàlisi . . . 15
5.5 Limitacions . . . 17
5.6 Conclusions . . . 18
5.7 Fórmules complementàries . . . 18

6 Problema d'Uriel: classificació de materials elèctrics amb DBSCAN **18**
6.1 Codi font . . . 20

7 Conclusions de l'equip **23**

Annex: codi complet del problema d'Arnau **25**

1 Equip i repartiment dels problemes

Exposició del tema «Aprenentatge no supervisat i detecció d'anomalies». L'objectiu és entendre com trobar estructura i patrons en dades sense etiquetar amb clustering (K-Means, DBSCAN, jeràrquic), mescles gaussianes i Isolation Forest, i com avaluar un clustering sense etiquetes (silhouette i mètode del colze).

Integrant	Problema	Algorisme principal	Fitxer
Arnau Montull	Detecció d'explosions nuclears entre sismes (catàleg USGS)	DBSCAN (haversine)	`nukeDBSCAN_arnau.py`
Matías	Agrupació dels sismes de l'últim mes i detecció d'anomalies	K-Means ($K = 3$) i k veïns més propers	`presentation_matias.py`, `validacio.py`
Martí	Expressió gènica (leucèmia ALL/AML): agrupar pacients i mòduls de gens sense etiquetes	Clustering jeràrquic (single, complete, average) i K-Means	`clustering_marti.py`
Uriel	Classificació de materials elèctrics (conductors, semiconductors i aïllants)	DBSCAN ($\varepsilon = 0.65$, $v_{\min} = 3$) i Z-Score manual	`materiales_uriel.py`

Taula 1: Repartiment dels problemes. Cada fitxer es diu `<nemotècnic>_<estudiant>.py`.

2 Fonaments comuns

Resum dels conceptes que fan servir els problemes.

Aprenentatge no supervisat i clustering. Es busca estructura en dades sense etiquetes. Cada algorisme necessita una mesura de similitud (distància euclidiana, haversine, correlació...) i l'elecció condiciona el resultat.

K-Means. Divideix les dades en K grups minimitzant la suma de distàncies quadràtiques de cada punt al centroide del seu grup, $J = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$. Es resol amb l'algorisme de Lloyd (assignar cada punt al centroide més proper i recalcular els centroides, fins a convergir) [16, 17], inicialitzat amb K-means++ [18], que tria els centroides inicials lluny els uns dels altres. És un clustering *dur*, suposa grups compactes i de mida semblant, i cal fixar K.

Com triar K sense etiquetes. Mètode del colze (la inèrcia deixa de baixar de manera apreciable), coeficient de silueta, $s = (b - a)/\max(a, b)$, amb a la distància mitjana als punts del seu grup i b la distància mitjana al grup veí més proper, i índex de Davies–Bouldin.

DBSCAN. Agrupa per densitat: un punt és *central* si té com a mínim $v_{\min}$ punts (ell inclòs) a distància $\leq \varepsilon$; els grups són les components connexes de punts centrals i els punts que no abasta cap

queden com a soroll (-1). No cal fixar el nombre de grups i troba formes arbitràries.

Clustering jeràrquic, mescles gaussianes i Isolation Forest. El jeràrquic construeix un dendrograma segons el tipus d'enllaç (simple, mitjà, Ward) i es talla a una altura. Les mescles gaussianes (`GaussianMixture`) donen una assignació tova per probabilitat. Isolation Forest [20] puntua com a anòmals els punts que s'aïllen amb pocs talls aleatoris. Les mescles gaussianes i Isolation Forest formen part del temari i s'expliquen a l'exposició, però cap dels quatre problemes no els aplica: en tots ells la detecció d'anomalies es fa amb DBSCAN (punts de soroll o clústers atípics) i amb la distància al k-èsim veí. **[PENDENT: l'equip revisa i amplia aquest resum amb el que s'exposi.]**

3 Problema d'Arnau Montull: detecció d'explosions nuclears amb DBSCAN

3.1 Interès del problema

El desenvolupament de l'armament nuclear ha donat lloc a un període de pau sense precedents entre les grans potències. La possibilitat d'una destrucció mútua assegurada ha dissuadit qualsevol guerra a gran escala entre potències nuclears des de la Segona Guerra Mundial. Alhora, aquestes potències han procurat que el nombre d'estats amb accés a aquesta tecnologia sigui el més reduït possible, i ho han fet mitjançant tractats com el de No Proliferació (1968) i el Tractat de Prohibició Completa dels Assaigs Nuclears (CTBT, 1996), que encara no ha entrat en vigor.

Els fets recents mostren que aquest equilibri és fràgil. L'enriquiment d'urani per part de l'Iran, que s'acosta a l'ús militar, ha alarmat Occident i ha desencadenat un conflicte armat a l'Orient Mitjà entre l'Iran, d'una banda, i Israel i els Estats Units, de l'altra, amb implicació dels seus aliats del golf Pèrsic.

L'últim estat que ha entrat al club nuclear és Corea del Nord, que va fer el seu primer assaig el 2006, fa pocs anys en termes històrics. Amb l'arma, Pyongyang ha blindat el seu règim autoritari davant de qualsevol intent d'intervenció exterior. Corea del Nord va anunciar els seus assaigs, però les xarxes sísmiques globals els van registrar de manera independent i en van permetre la localització i l'estimació de la mida [13]. Aquesta capacitat de verificar a distància, només amb dades sísmiques, és la que dona sentit als tractats anteriors. En aquest treball en provem una versió simple: veure si un algorisme no supervisat pot assenyalar les explosions d'un catàleg sísmic sense llegir-ne l'etiqueta.

3.2 Plantejament de la solució

Els sismògrafs registren igual terratrèmols i explosions nuclears. Utilitzem el catàleg de l'USGS i només dues variables de cada esdeveniment: la posició i la profunditat. L'agrupament (DBSCAN) és no supervisat i la finestra de profunditat es fixa amb un criteri físic, sense optimitzar-la contra l'etiqueta; aquesta només s'utilitza per avaluar (vegeu les limitacions). La idea física és doble: les proves nuclears es repeteixen als mateixos polígons d'assaig (Nevada, Semipalatinsk, Nova Zembla, Lop Nur, Mururoa...), de manera que formen agrupacions espacials molt compactes, i passen gairebé a la superfície, mentre que els terratrèmols del catàleg tenen una profunditat mediana de 33 km i només el 0,25 % és a menys de 2,5 km.

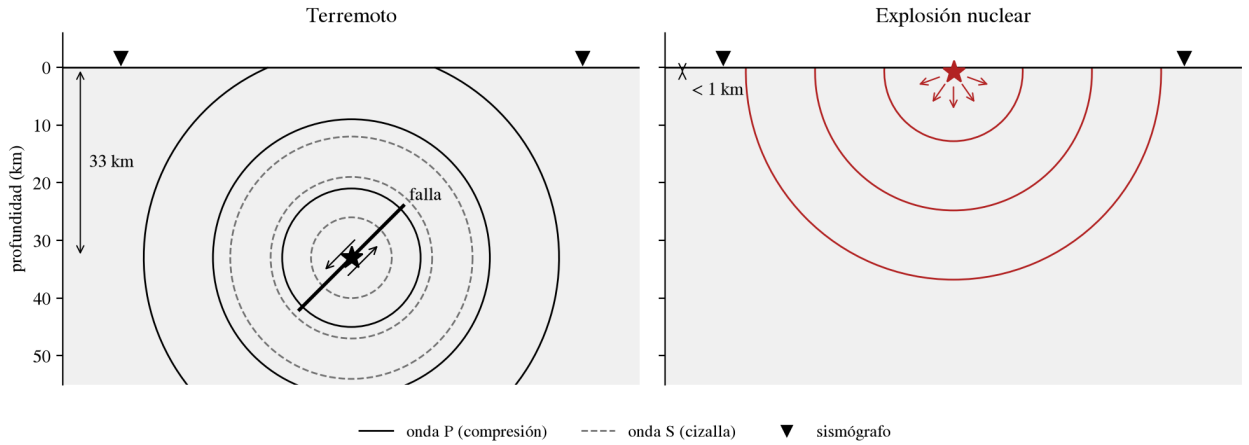


Figura 1: Tall vertical esquemàtic. Un terratrèmol s'origina quan una falla llisca en profunditat (mediana del catàleg, 33 km) i emet ones P i S. Una explosió nuclear subterrània és un pols de pressió gairebé a la superfície i emet sobretot ones P. Els sismògrafs (triangles) registren tots dos esdeveniments; aquí només es fa servir la profunditat del focus que dona el catàleg.

És un problema d'aplicació física que no figura entre els suggeriments de l'enunciat. Les dades són públiques i el treball és de detecció i verificació, no una guia per amagar proves.

3.3 Dades

Catàleg de l'USGS (*ComCat*) amb tots els esdeveniments de magnitud $M \geq 4{,}5$ entre 1951 i 2025 (`datos/sismos_usgs_M45_1951_2025.csv`), descarregat amb `nukedatos.py`. No es filtra per tipus d'esdeveniment. Després de treure els esdeveniments sense latitud, longitud, profunditat o magnitud en queden 298 127: 614 etiquetats per l'USGS com a `nuclear explosion`, 297 413 `earthquake` i 100 d'altres tipus (71 erupcions volcàniques, 16 `explosion`, 6 esfondraments de mina, 3 esclats de roca, 2 esllavissades, 1 voladura de pedrera i 1 `other event`). La columna `tipo` *només s'utilitza per avaluar*, mai per entrenar.

`nukedatos.py` demana a l'API de l'USGS un any cada vegada (cada consulta torna com a màxim 20 000 esdeveniments), torna a provar-ho si falla la xarxa i desa el resultat canviant el nom de la columna `type` per `tipo`. El codi complet és a l'Annex.

```
def descargar_year(year):
    # La API limita cada consulta a 20 000 eventos, asi que se pide un año cada vez
    consulta = f'{URL}&starttime={year}-01-01&endtime={year + 1}-01-01'

    for intento in range(4):
        try:
            return pd.read_csv(consulta)
        except Exception:
            print(f'  {year}: reintento')

    raise RuntimeError(f'No se pudo descargar {year}')
```

Codi 1: `nukedatos.py`: descàrrega d'un any del catàleg, amb reintents.

`nukeDBSCAN_arnau.py` carrega el CSV i es queda amb els esdeveniments que tenen posició, profunditat i magnitud.

```
# --- Datos ---
df = pd.read_csv(ARCHIVO)
df = df.dropna(subset=['latitude', 'longitude', 'depth', 'mag'])
df = df[df['mag'] >= 4.5]
df = df.reset_index(drop=True)
```

Codi 2: `nukeDBSCAN_arnau.py`: càrrega i neteja de les dades.

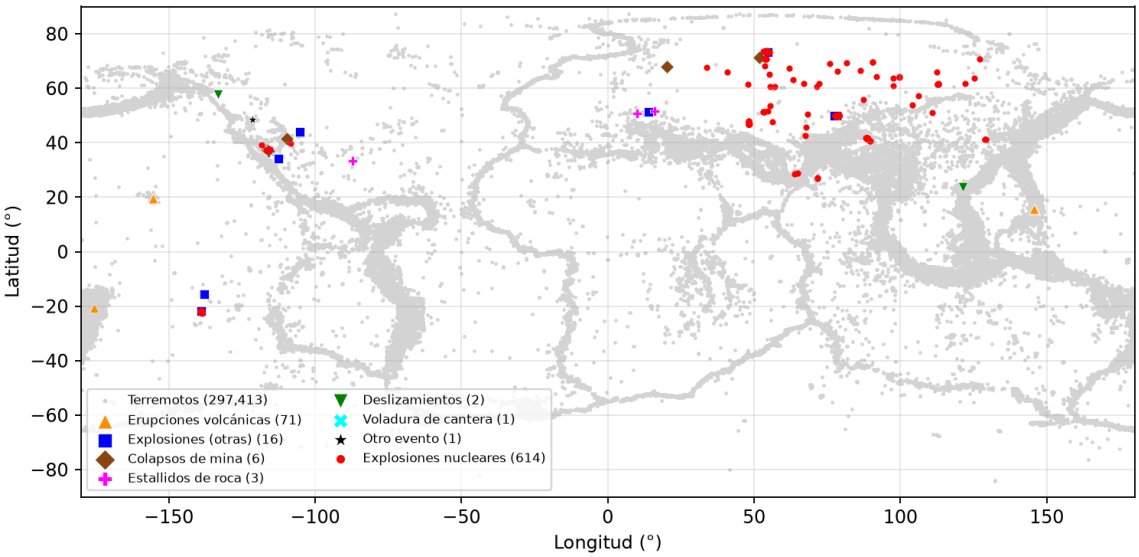


Figura 2: Catàleg complet: les explosions nuclears (vermell) es concentren en uns pocs polígons d'assaig. Els 100 esdeveniments d'altres tipus de l'USGS (erupcions volcàniques, altres explosions, esfondraments de mina, esclats de roca, esllavissades, voladura de pedrera i un altre esdeveniment) porten cadascun el seu color i marcador.

3.4 Fonamentació teòrica

DBSCAN [1] defineix els grups per densitat. Donats ε i $v_{\min}$, un punt és central si el seu veïnat de radi ε conté com a mínim $v_{\min}$ punts; dos punts centrals a distància $\leq \varepsilon$ pertanyen al mateix grup (accessibilitat per densitat) i els punts sense prou veïnat que no abasta cap punt central són soroll. Amb $v_{\min} = 1$ tot punt és central i els grups són les components connexes del graf que uneix els sismes a menys de ε; és equivalent al clustering jeràrquic d'enllaç simple tallat a l'altura ε. Schubert et al. [19] revisen com triar ε i $v_{\min}$ i per què valors baixos de $v_{\min}$ apropen DBSCAN a l'enllaç simple; per a ε, el mateix article original proposa el gràfic de distàncies al k-èsim veí ordenades, que es fa servir a la validació.

Els epicentres són sobre l'esfera, així que la distància correcta és l'ortodròmica. La fórmula de *haversine* [3],

$$d = 2R \arcsin \sqrt{\sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}}, \qquad R = 6371 \text{ km},$$

és numèricament estable per a distàncies petites i està implementada a `sklearn.cluster.DBSCAN` [2] (`metric='haversine'`, amb ε en radiants, $\varepsilon_{\text{rad}} = \varepsilon_{\text{km}}/R$). Aquí el clustering només agrupa per posició; el criteri físic (la profunditat) és el que etiqueta cada grup.

Com que les classes estan molt desequilibrades (614 explosions davant de gairebé 300 000 terratrèmols), l'exactitud no serveix: un detector que no marqués mai res encertaria el 99,8 %. S'utilitzen la precisió $P = \text{TP}/(\text{TP}+\text{FP})$, el recall $R = \text{TP}/(\text{TP}+\text{FN})$ i la seva mitjana harmònica, $F_1 = 2PR/(P+R) = 2\,\text{TP}/(2\,\text{TP}+\text{FP}+\text{FN})$.

3.5 Mètode: com es detecta

1. **Agrupar per posició.** DBSCAN amb distància *haversine* (distància sobre l'esfera; les coordenades es passen a radiants i el radi ε es divideix pel radi terrestre, 6371 km). Dos sismes són del mateix clúster si hi ha una cadena de sismes a menys de ε km l'un del següent.

2. **Mirar la profunditat de cada clúster.** Es calcula la *mediana* de la profunditat de cada clúster (la mediana no es deixa enganyar per uns pocs valors atípics).
3. **Marcar com a sospitosos** els clústers amb la mediana dins de $[d_{\min}, d_{\max})$: gairebé a la superfície. Tots els sismes d'un clúster sospitós es marquen com a *detectats*.
4. **Avaluar contra l'etiqueta.** Encert: detectat i explosió. Fals positiu (FP): detectat però no és una explosió. Fals negatiu (FN): explosió no detectada.

El pas 1 és la funció `agrupar`: passa les coordenades a radiants, divideix ε pel radi terrestre i crida DBSCAN amb la mètrica *haversine*.

```
# --- Agrupar los sismos cercanos con DBSCAN ---
def agrupar(df, eps_km, vmin):
    radio_tierra_km = 6371
    coordenadas = np.radians(df[['latitude', 'longitude']])
    eps_rad = eps_km / radio_tierra_km

    dbscan = DBSCAN(eps=eps_rad, min_samples=vmin, metric='haversine')
    cluster = dbscan.fit_predict(coordenadas)
    return pd.Series(cluster, index=df.index)
```

Codi 3: `agrupar`: DBSCAN amb distància haversine.

Els passos 2 i 3 són `nukeDBSCAN`: la mediana de la profunditat de cada clúster i la selecció dels que cauen dins de la finestra.

```
# --- Marcar los sismos sospechosos de ser explosiones ---
def nukeDBSCAN(df, eps_km, vmin, dmin, dmax):
    cluster = agrupar(df, eps_km, vmin)

    # Profundidad mediana de cada cluster (el ruido, -1, no cuenta)
    sin_ruido = cluster != -1
    mediana = df['depth'].groupby(cluster[sin_ruido]).median()

    # Sospechoso: cluster casi en la superficie, con la mediana entre dmin y dmax
    es_superficial = (mediana >= dmin) & (mediana < dmax)
    clusters_sospechosos = mediana[es_superficial].index

    return cluster.isin(clusters_sospechosos)
```

Codi 4: `nukeDBSCAN`: clústers sospitosos per la mediana de la profunditat.

El pas 4 és `errores`, que compta els FP i els FN amb l'etiqueta `tipo`.

```
# --- Falsos positivos y falsos negativos ---
def errores(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'

    # Falso positivo: detectado, pero no es una explosion
    falsos_positivos = (detectado & ~es_explosion).sum()
    # Falso negativo: es una explosion, pero no se detecta
    falsos_negativos = (~detectado & es_explosion).sum()
    return falsos_positivos, falsos_negativos
```

Codi 5: `errores`: falsos positius i falsos negatius.

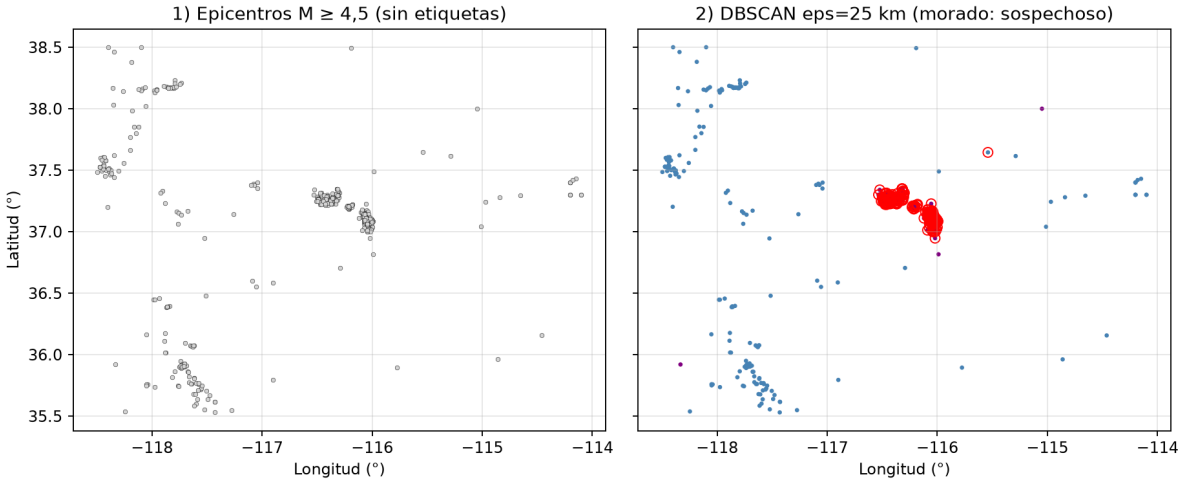


Figura 3: Zoom al polígon de Nevada. Esquerra: epicentres sense etiquetar. Dreta: clústers de DBSCAN; en morat, els clústers sospitosos per profunditat; cercles vermells: explosions reals (etiqueta de l'USGS).

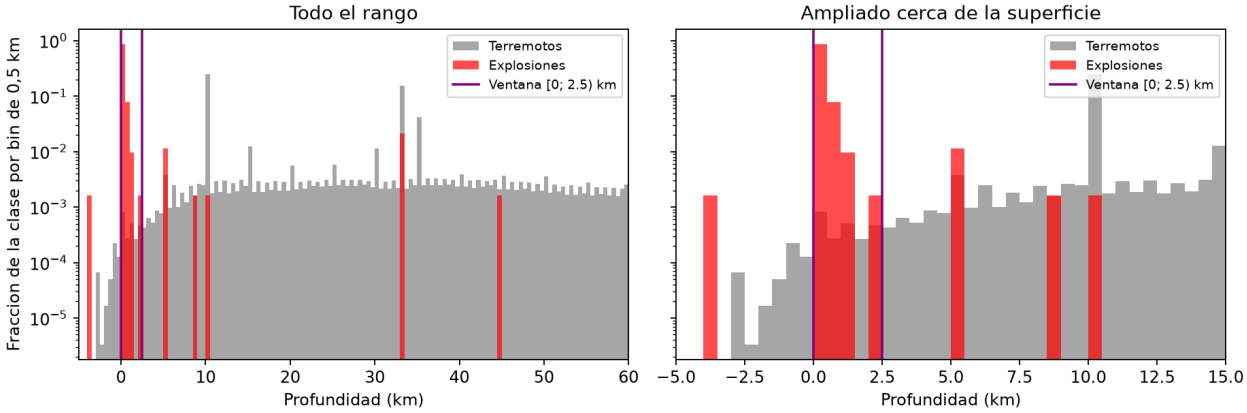


Figura 4: Distribució de profunditats: fracció de cada classe en intervals de 0,5 km (escala logarítmica i mateix eix vertical), completa i ampliada a prop de la superfície. Les línies morades delimiten la finestra $[d_{\min}, d_{\max})$, que s'aplica a la *mediana* de cada clúster i no a cada sisme.

3.6 Elecció dels paràmetres

Paràmetres fixos: $\varepsilon = 25$ km, $v_{\min} = 1$, $d_{\min} = 0$ i $d_{\max} = 2{,}5$ km. Són en un únic diccionari al principi de `nukeDBSCAN_arnau.py`, que `nukeval.py` importa.

```
# Parametros fijos
#   eps_km: radio de DBSCAN. vmin: sismos minimos por cluster.
#   dmin, dmax: una explosion es casi superficial, asi que un cluster es sospechoso
#   si la mediana de su profundidad esta entre 0 y 2.5 km.
parametros = dict(eps_km=25, vmin=1, dmin=0, dmax=2.5)
```

Codi 6: `nukeDBSCAN_arnau.py`: paràmetres fixos.

- **Finestra de profunditat** [0; 2,5) **km:** criteri físic. Una explosió nuclear soterrada és a menys d'uns pocs km de la superfície; els terratrèmols tectònics són, majoritàriament, molt més profunds.
- $\varepsilon = 25$ **km:** de l'ordre de la mida d'un polígon d'assaig. És el valor amb menys falsos positius dels provats (de 5 a 100 km, vegeu la validació): amb menys radi els terratrèmols superficials

queden en grups solts i amb més radi els grups d'explosions es fonen amb terratrèmols més profunds. Aquí sí que es mira l'etiqueta.

- $v_{\min} = 1$**:** amb $v_{\min} = 3$ empitjora (vegeu les limitacions).

L'script `nuke_optimizar.py` (cerca de la finestra i de ε per F_1) es va provar, però no es fa servir per als paràmetres finals: el seu òptim, [0; 0,1) km, queda a la vora de la seva graella i aprofita que l'USGS registra amb profunditat exactament 0,0 km el 85 % de les explosions (521 de 614) i només 154 dels 297 513 esdeveniments que no són explosions nuclears. Això mesura com cataloga l'USGS i no la física. La taula compara finestres amb $\varepsilon = 25$ km.

Finestra (km)	TP	FP	FN	F_1
[0; 0,1) (la que triava l'optimitzador)	585	58	29	0,931
[0; 2,5) (la utilitzada)	586	115	28	0,891
[0; 5)	587	363	27	0,751
[0; 10)	588	2 897	26	0,287

Taula 2: Efecte de la finestra de profunditat ($\varepsilon = 25$ km, $v_{\min} = 1$).

3.7 Validació de ε i estabilitat (`nukeval.py`)

Seguint l'esquema de validació del temari (silueta, colze i estabilitat), `nukeval.py` avalua ε amb $v_{\min} = 1$ i la finestra [0; 2,5) km. Per a cada ε compta els clústers, calcula la silueta amb distància haversine sobre una mostra de 2 000 sismes (sense etiquetes) i els FP i FN (amb etiquetes).

```
# --- Silueta de unos clusters (sobre una muestra de 2000 sismos) ---
def silueta(df, cluster):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    try:
        return silhouette_score(coordenadas, cluster, metric='haversine',
                                sample_size=2000, random_state=42)
    except ValueError:
        # La muestra no tiene al menos 2 clusters
        return np.nan


# --- Para cada eps: numero de clusters, silueta, FP y FN ---
def evaluar_eps(df, vmin, dmin, dmax):
    print('\n--- ELECCION DE EPS ---')
    print(f"{'eps (km)':<10}{'clusters':<10}{'silueta':<10}{'FP':<8}{'FN':<8}")
    print('-' * 46)

    n_clusters = []
    siluetas = []
    for eps in EPS_PROBADOS:
        cluster = agrupar(df, eps, vmin)
        detectado = nukeDBSCAN(df, eps, vmin, dmin, dmax)
        fp, fn = errores(df, detectado)

        n_clusters.append(cluster.nunique())
        siluetas.append(silueta(df, cluster))
        print(f'{eps:<10}{n_clusters[-1]:<10}{siluetas[-1]:<10.3f}{fp:<8}{fn:<8}')

    return n_clusters, siluetas
```

Codi 7: `nukeval.py`: silueta d'uns clústers i taula per a cada ε provat.

L'estabilitat compara els clústers del catàleg complet amb els de cinc mostres aleatòries del 50 % mitjançant l'ARI.

```
# --- Estabilidad: ARI entre los clusters del catalogo y los de una muestra del 50% ---
```

```python
def evaluar_estabilidad(df, eps_km, vmin):
    print(f'\n--- ESTABILIDAD (eps={eps_km} km, muestra del 50%) ---')
    cluster_completo = agrupar(df, eps_km, vmin)

    for semilla in [1, 2, 3, 4, 5]:
        muestra = df.sample(frac=0.5, random_state=semilla)
        cluster_muestra = agrupar(muestra, eps_km, vmin)

        ari = adjusted_rand_score(cluster_completo[muestra.index], cluster_muestra)
        print(f'Semilla {semilla}: clusters={cluster_muestra.nunique()}, ARI={ari:.4f}')
```

Codi 8: `nukeval.py`: estabilitat amb l'Adjusted Rand Index.

ε (km)	Clústers	Silueta	FP	FN
5	111 117	0,01	500	15
10	44 804	−0,18	251	13
25	11 049	−0,45	115	28
40	5 691	−0,43	123	30
50	4 214	−0,50	483	28
100	1 786	−0,51	151	269

Taula 3: Validació de ε (extracte; `nukeval.py` n'imprimeix 10 valors).

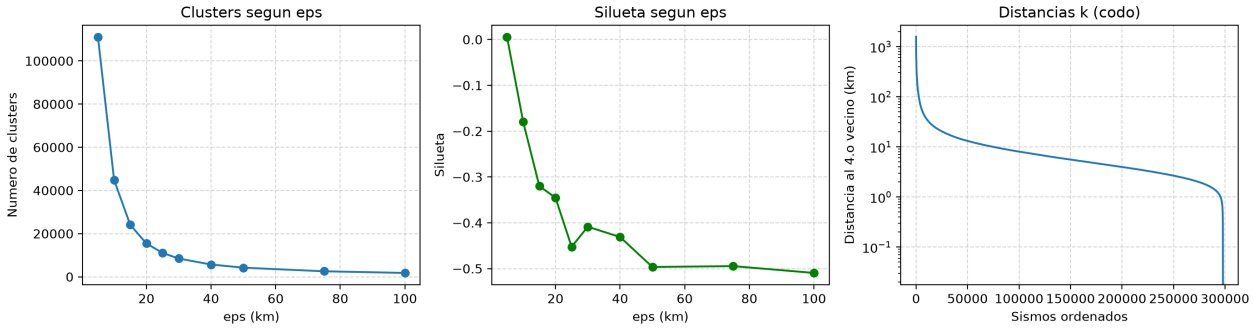


Figura 5: Clústers, silueta i distància al 4t veí (colze clàssic de DBSCAN) en funció de ε.

- **La silueta no tria ε:** és negativa i decreix quan ε augmenta. Amb distància per densitat, DBSCAN uneix cadenes allargades de sismes i la silueta [8], que premia grups compactes i separats, el penalitza. Tampoc el gràfic de distàncies al 4t veí no mostra un colze clar entre 20 i 30 km.
- **El que sí que limita ε és la profunditat:** els FP baixen quan ε creix fins a un mínim de 115 a 25 km (amb ε petit molts terratrèmols superficials queden en grups solts); a partir de 40–50 km els grups d'explosions es fonen amb terratrèmols més profunds: els FP pugen a 483 amb 50 km i els FN passen de 28 a 261 amb 75 km, perquè la mediana deixa de ser superficial.
- **Estabilitat:** l'ARI [9] (val 1 si dues particions coincideixen i 0 si s'assemblen com dues a l'atzar) entre el clustering de tot el catàleg i el d'una mostra aleatòria del 50 % (5 llavors, $\varepsilon = 25$ km) val entre 0,54 i 0,65: moderat, perquè en treure sismes es trenquen cadenes de DBSCAN.

3.8 Resultats

Amb els paràmetres fixos: 586 encerts, 115 FP i 28 FN, és a dir, precisió del 83,6 %, recall del 95,4 % i $F_1 = 0{,}891$.

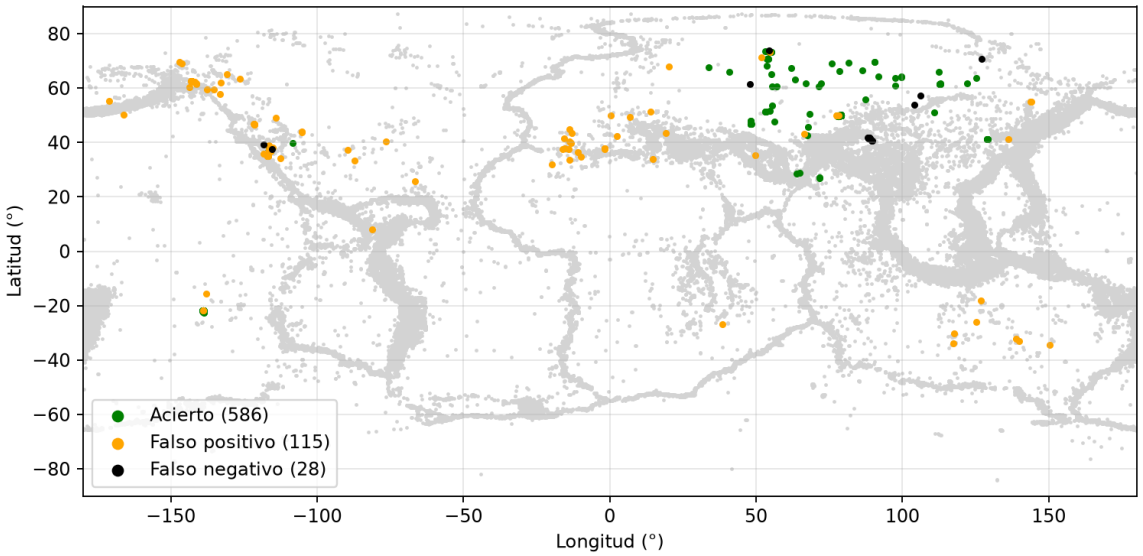


Figura 6: Resultat final: encerts (verd), falsos positius (taronja) i falsos negatius (negre).

3.9 Què hi ha darrere de cada clúster

DBSCAN no en sap res, d'història, però els clústers que troba són els polígons d'assaig de la Guerra Freda. Les 614 explosions cauen en 59 clústers; la taula mostra els més grans, amb el lloc que dona el mateix catàleg de l'USGS.

Polígon	Anys	Explosions	Esdeveniments	Mediana (km)	Detectat?
Nevada (EUA)	1962–1992	230	256	0,0	Sí
Semipalatinsk, Balapan	1973–1989	92	92	0,0	Sí
Semipalatinsk, Degelen	1973–1989	86	87	0,0	Sí
Mururoa (Polinèsia Francesa)	1977–1996	74	76	0,0	Sí
Nova Zembla, Matotxkin Xar	1973–1990	20	21	0,0	Sí
Lop Nur (Xina)	1976–1996	18	47	15,7	No
Astracan (Rússia)	1980–1984	15	15	0,0	Sí
Punggye-ri (Corea del Nord)	2009–2017	5	5	0,0	Sí

Taula 4: Clústers de DBSCAN amb més explosions ($\varepsilon = 25$ km, $v_{\min} = 1$).

- **Nevada.** És el polígon on els Estats Units van fer la majoria dels seus assajos: 928 (100 atmosfèrics i 828 subterranis) segons la llista oficial del Departament d'Energia [5]; des del juliol de 1962 tots van ser subterranis. Al catàleg ($M \geq 4{,}5$) en queden 230, tan junts que DBSCAN els encadena en un únic clúster. També és el que aporta més falsos positius: els 26 esdeveniments del clúster que no són explosions nuclears (16 terratrèmols, 6 `explosion`, 3 esfondraments i 1 voladura) es marquen amb ell. Després d'un assaig subterrani, el sostre de la cavitat es pot enfonsar i aquest esfondrament genera el seu propi senyal sísmic, de tipus implosiu [15]; alguns d'aquests esdeveniments podrien ser-ho, o bé proves que l'USGS va registrar amb una altra etiqueta.
- **Semipalatinsk.** Al polígon soviètic del Kazakhstan s'hi van fer 456 explosions nuclears fins al 1989 [11]. Apareix partit en dos clústers separats per uns 60 km: Balapan, amb assajos en pous verticals, i el massís de Degelen, amb assajos en galeries. Són 178 explosions amb mediana de profunditat 0 i gairebé cap terratrèmol al voltant: el cas més fàcil.
- **Mururoa.** França va fer 147 assajos subterranis entre 1975 i 1996, 137 a Mururoa i 10 a Fangataufa [12]. Lluny de qualsevol zona sísmica, el clúster de Mururoa conté 74 explosions i només 2 esdeveniments més.

- **Nova Zembla.** L'arxipèlag àrtic rus dona dos clústers petits: Matotxkin Xar, la zona central d'assajos subterranis en galeries, i Txernaia Guba, al sud. Més al nord, sobre la península de Sukhoi Nos, va esclatar el 30 d'octubre de 1961 la *Bomba Tsar*, d'uns 50 Mt, l'assaig més gran de la història [11]. Va ser una explosió aèria a uns 4 km d'altura i l'USGS la registra a -4 km, per sobre de la superfície: queda fora de la finestra [0; 2,5) i és un dels 28 falsos negatius.
- **Lop Nur.** La Xina hi va fer els seus 45 assajos entre 1964 i 1996 [11], al costat del Tian Shan, una serralada amb molta sismicitat natural. És la gran fallada del mètode: les 18 explosions del clúster cauen al costat de 29 terratrèmols, amb mediana de 24,5 km, i diverses explosions figuren a 33 km, una profunditat fixa que el catàleg feia servir quan no la podia resoldre. La mediana del clúster puja a 15,7 km i es perd tot el grup: són 18 dels 28 FN. Dels 28, 10 tenen aquesta profunditat de 33 km.
- **Astracan i Iacútia.** Clústers petits a Rússia sense cap polígon militar. Corresponen a explosions nuclears amb finalitats civils del programa soviètic; les 15 d'Astracan (1980–1984) van crear cavitats en doms de sal per emmagatzemar gas.
- **Punggye-ri.** Els cinc assajos nord-coreans amb $M \geq 4{,}5$ (2009–2017); el del 2006 va tenir una magnitud inferior i queda per sota del tall. El del 3 de setembre de 2017, de magnitud 6,3, és l'explosió més gran del segle XXI al catàleg. Uns 8,5 minuts després, el sostre de la cavitat es va enfonsar i va produir un segon esdeveniment de magnitud propera a 4 [13], que no entra al nostre catàleg per magnitud.
- **Un assaig amagat entre terratrèmols.** El 26 d'octubre de 1963, els Estats Units van fer l'assaig *Shoal*, de 12 kt, en granit a prop de Fallon (Nevada), lluny del polígon [5]. Formava part del programa Vela Uniform, que estudiava com detectar assajos subterranis, i es va fer en una zona amb terratrèmols reals: la seqüència del 1954 de Rainbow Mountain, Fairview Peak i Dixie Valley [14]. DBSCAN l'agrupa amb aquests terratrèmols i la mediana deixa de ser superficial: el mètode falla just on l'explosió s'assembla a la sismicitat natural.

Limitacions i honestedat del resultat.

- **Amb $v_{\min} = 1$ no hi ha soroll:** cada sisme aïllat forma el seu propi clúster. Aquí DBSCAN actua com un agrupador per proximitat encadenada, i la selecció real la fa el filtre de profunditat. Amb $v_{\min} = 3$ el resultat empitjora ($F_1 = 0{,}899$; FN = 69), perquè moltes proves aïllades es perdrien com a «soroll».
- **Enfocament híbrid, no purament no supervisat:** DBSCAN agrupa sense etiquetes i la finestra de profunditat és un criteri físic previ, però ε s'ha triat mirant l'F_1 contra l'etiqueta entre uns pocs valors, perquè ni la silueta ni el colze no el determinen (vegeu la validació).
- **Generalització:** no hi ha cap subconjunt de prova separat: s'ajusta i s'avalua en el mateix catàleg.
- **Convenció del catàleg:** el 85 % de les explosions figura amb profunditat exactament 0,0 km. Una finestra més estreta, [0; 0,1), donaria $F_1 = 0{,}931$ però mesuraria aquesta convenció de l'USGS i no la física.
- **Falsos positius:** terratrèmols molt superficials (anys 80 a Washington, Califòrnia i Alaska) i altres esdeveniments no sísmics. Dels 115 FP, 24 són d'un altre tipus: 16 `explosion`, 5 esfondraments de mina, 1 voladura de pedrera, 1 esclat de roca i 1 esllavissada; cap de les 71 erupcions volcàniques no es marca. Si es descartessin els altres 100 esdeveniments no sísmics (només terratrèmols i explosions nuclears), serien 91 FP i $F_1 = 0{,}908$. Algunes proves oficials de Nevada figuren a l'USGS com a `earthquake`; aquest error és de l'etiqueta, no del mètode.
- **Biaix d'època:** no es fan servir l'any ni el nombre d'estacions com a variables, perquè delaten l'època del catàleg i no la física.

3.10 Conclusions

- Un clustering espacial sense etiquetes, combinat amb la profunditat mediana de cada grup, recupera el 95 % de les explosions del catàleg ($F_1 = 0{,}891$) amb 115 falsos positius entre gairebé 300 000 esdeveniments.
- El senyal útil és doble i físic: repetició als mateixos polígons d'assaig i profunditat gairebé nul · la.
- Línies de millora: triar ε en un subconjunt i avaluar-lo en un altre; afegir la magnitud i la forma d'ona com a variables; i fer servir la llista oficial DOE/NV-209 [5] en lloc de l'etiqueta de l'USGS per a les proves de Nevada.

3.11 Com reproduir-ho

1. Crear l'entorn: `python -m venv .venv` i instal · lar `requirements.txt` (numpy, pandas, matplotlib, scikit-learn, plotly).
2. Opcional: `python problemas/2_Nuclear/nukedatos.py` descarrega de l'USGS el catàleg (`sismos_usgs_M45` a `datos/`; si el fitxer ja existeix, no fa res.
3. `python problemas/2_Nuclear/nukeDBSCAN_arnau.py` (es pot executar des de qualsevol carpeta). Imprimeix els FP i FN de l'execució amb els paràmetres fixos i escriu `globo.html` (globus interactiu amb 4 vistes, funciona sense internet).
4. `python problemas/2_Nuclear/nuke_optimizar.py` reprodueix la cerca per F_1 que es va descartar (opcional; uns minuts).
5. `python problemas/2_Nuclear/nukeval.py` reprodueix la validació de ε i l'estabilitat (uns 4 min) i desa `nukeval_eps.png` i `nukeval_mapa.png`.
6. Les figures d'aquest document: `python docs/figuras_nuclear.py`.

4 Problema de Matías: agrupació de sismes i detecció d'anomalies

Material a `problemas/3_Sismes_Matias/` (README, codi, dades i informe propi en català, `informe_sismes.tex`). El que segueix resumeix el seu `README.md` i el seu codi; *no s'ha executat el seu codi ni s'han comprovat els resultats.*

Objectiu. Segmentar els sismes globals de l'últim mes amb K-Means ($K = 3$) segons la magnitud, la profunditat, la latitud i la longitud, per identificar patrons de subducció i activitat a les vores de placa, i detectar esdeveniments excepcionals (magnitud alta o profunditat inusual) com a anomalies.

Dades. `earthquakes.csv`: 10 174 esdeveniments del catàleg de l'USGS amb capçalera estàndard (`time, latitude, longitude, depth, mag, ...`); el primer és del 15/10/2026.

Mètode.

1. Càrrega, reordenació de columnes i filtratge de registres incomplets o invàlids.
2. Normalització z-score de les quatre variables (mitjana 0, desviació 1).
3. K-Means amb $K = 3$.
4. Anomalies: distància euclidiana mitjana als $k = 5$ veïns més propers en l'espai normalitzat; són anòmals els punts més aïllats.

Validació. Mètode del colze (WCSS), coeficient de silueta i *Adjusted Rand Index* (ARI) entre inicialitzacions diferents per mesurar l'estabilitat; `validacio.py` també genera el mapa geogràfic.

Reproduir. `pip install -r requirements.txt` (pandas, numpy, scikit-learn, matplotlib) i després `python presentation_matias.py` i `python validacio.py`. Hi ha versions `*_PANDAS.py` i `*_FINAL.py` dels dos scripts. Nota: el README esmenta `problema_matias.py`, que no és a la carpeta, i els fitxers no segueixen el format `<nemotècnic>_<estudiant>.py` que demana l'enunciat.

[PENDENT: Matías: resultats (valor de K triat, silueta, ARI, anomalies trobades) i la seva conclusió.]

5 Problema de Martí: expressió gènica i clustering jeràrquic (leucèmia ALL/AML)

Material a `problemas/4_RNAseq_Marti/` (codi `clustering_marti.py`, proves `test_clustering.py`, `README.txt`, figures i informe propi `informe_curt.tex`). El text que segueix és l'informe de Martí tal com l'ha escrit ell.

Resum. Aquesta implementació aplica l'aprenentatge no supervisat al conjunt de dades de leucèmia de Golub et al. (72 pacients, 7.129 gens) per comprovar, primer, si és possible recuperar sense etiquetes la distinció entre leucèmia limfoblàstica aguda (ALL) i mieloide aguda (AML) i, segon, quins grups de gens mostren patrons d'expressió semblants. Després d'un preprocessament logarítmic i d'una selecció dels 500 gens més variables, s'agrupen els pacients amb clustering jeràrquic (tres tipus d'enllaç) i K-Means, i els gens amb clustering jeràrquic de distància de correlació. Els diagnòstics clínics s'usen només per validar els models. K-Means assoleix un ARI de 0,89 i l'enllaç *complete* de 0,79, mentre que el coeficient de Silhouette indica $k = 2$ com a nombre òptim de grups.

5.1 Plantejament del problema

Les cèl · lules fabriquen proteïnes a partir de la informació dels gens, i ho fan a través de l'ARN missatger (mRNA): com més activitat té un gen, més mRNA se'n detecta. Aquesta quantitat s'anomena *nivell d'expressió*. Tècniques com el microarray d'oligonucleòtids (usat a [6]) o la seqüenciació d'ARN la mesuren per a milers de gens alhora, de manera que cada pacient queda descrit per un vector amb els nivells d'expressió de tots els gens mesurats. Les leucèmies agudes limfoblàstica (ALL) i mieloide (AML) deriven de línies cel · lulars diferents de la sang i requereixen tractaments diferents; per això distingir-les bé és clínicament rellevant.

El problema és naturalment no supervisat per dos motius. Primer, no sabem a priori quins gens funcionen junts: els gens que s'expressen amb patrons semblants entre pacients (gens co-expressats) sovint estan regulats conjuntament o participen en processos biològics comuns [7], però aquesta informació no ve etiquetada. Segon, volem comprovar si l'estructura de les dades, per si sola, reprodueix el diagnòstic clínic sense haver-lo vist durant el modelatge. A més, es treballa amb alta dimensionalitat (7.129 gens per només 72 pacients), una situació on les distàncies són poc intuïtives i cal escollir amb cura la mètrica i el preprocessament.

Els objectius de la implementació són els següents:

1. Identificar les leucèmies ALL/AML sense etiquetes, tallant l'arbre o K-Means a $k = 2$, amb un ARI superior a 0,7.
2. Comparar tres criteris d'enllaç (*single*, *complete*, *average*) i K-Means, amb una mètrica interna (Silhouette) i una d'externa (ARI).
3. Identificar mòduls de gens co-expressats i triar-ne el nombre amb el coeficient de Silhouette.
4. Visualitzar conjuntament gens i pacients amb un mapa de calor (via dendrogrames).

Pel que fa a l'abast i els supòsits, s'usa un únic conjunt de dades, es treballa amb els 500 gens més variables, la distància és de correlació (o euclidiana), les etiquetes ALL/AML només s'usen després del clustering per validar, no s'elimina cap pacient i no s'interpreten biològicament els mòduls de gens.

Com a aportacions pròpies, el treball agrupa tant pacients com gens i els mostra junts, compara el mapa de calor brut amb el reordenat pels dendrogrames per veure què aporta el clustering, i corregeix un problema de preprocessament detectat en una primera versió (apartat de Metodologia).

5.2 Fonament teòric

Per comparar perfils d'expressió interessa la *forma* del patró i no la seva magnitud: dos gens que pugen i baixen igual entre pacients han de ser veïns encara que un tingui el doble d'amplitud. Per això s'usa la distància de correlació de Pearson, $d_r(u, v) = 1 - r(u, v)$, que val 0 si els perfils estan perfectament correlacionats i 2 si estan perfectament anticorrelacionats; combinada amb l'enllaç *average*, és l'opció clàssica en expressió gènica [7]. K-Means, en canvi, usa la distància euclidiana.

El clustering jeràrquic aglomeratiu parteix de cada element com a clúster propi i fusiona a cada pas els dos clústers més propers; el resultat és un arbre (dendrograma) que es pot tallar a l'altura desitjada per obtenir k grups. Els criteris d'enllaç difereixen en com mesuren la distància entre clústers:

- **Single:** distància mínima entre parells d'elements (sensible a punts aïllats i a l'efecte cadena).
- **Complete:** distància màxima entre parells; afavoreix clústers compactes.
- **Average:** distància mitjana entre tots els parells; equilibri entre els dos anteriors.

K-Means divideix les dades en k grups minimitzant la *inèrcia*, és a dir, la suma de quadrats de les distàncies de cada punt al centre del seu grup. Cal fixar k per endavant i el resultat depèn de la inicialització, per això es repeteix diverses vegades i es guarda la millor. El *mètode del colze* representa la inèrcia en funció de k i busca el punt on la millora deixa de ser apreciable.

Per validar sense etiquetes s'usa el *coeficient de Silhouette* [8], que mesura com de bé encaixa cada element i en el seu grup, $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \in [-1, 1]$, on $a(i)$ és la distància mitjana als altres elements del seu grup i $b(i)$ la del grup veí més proper; se'n tria el k de silueta mitjana màxima. Com a validació externa s'usa l'*Adjusted Rand Index* (ARI) [9], que compara dues particions comptant els parells d'elements que coincideixen i corregeix l'acord esperat per atzar: val 1 si són idèntiques, aproximadament 0 si equivalen a una partició aleatòria i pot ser lleugerament negatiu. Les fórmules completes són a l'Annex 5.7.

Finalment, un *clustermap* mostra la matriu d'expressió com un mapa de calor amb files (gens) i columnes (pacients) reordenades segons els seus dendrogrames, de manera que els blocs de color revelen gens co-expressats en grups de pacients [7].

5.3 Metodologia

Dades. S'ha utilitzat el conjunt de Golub et al. [6], descarregat d'OpenML (ID 1104): l'expressió de 7.129 gens, mesurada amb microarrays Affymetrix, en 72 pacients (47 ALL i 25 AML), organitzada en una matriu $X \in \mathbb{R}^{7129\times72}$. Les etiquetes clíniques no intervenen en cap etapa del modelatge. La matriu en brut no té valors nuls, va de -28.400 a 71.369 i és molt asimètrica (mitjana 619,78, mediana 120,00); els valors negatius provenen de la correcció del fons i no tenen sentit com a expressió.

Preprocessament. S'inspira en el de Golub et al. [6]. Primer, un retall de rang a [100, 16.000] (`np.clip`), que elimina els valors negatius i les fluctuacions per sota del límit de detecció i limita la saturació dels detectors. Després, la transformació $x' = \log_2(x+1)$, que redueix l'asimetria, estabilitza la variància i fa que les diferències es llegeixin com a canvis relatius d'expressió (*fold-change*); com que $x \geq 100$, el $+1$ és innecessari però inofensiu. Una primera versió retallava només a 0: concentrava més de 10^5 valors en un sol punt i feia aparèixer gens amb variàncies artificialment altes, que haurien dominat la selecció posterior. Finalment, es conserven els $N = 500$ gens de variància més alta, perquè

els gens gairebé constants no informen sobre diferències entre pacients (selecció sense etiquetes), i cada gen s'estandarditza amb el z-score, $z = (x - \mu)/\sigma$ entre pacients. El resultat és una matriu $X_{\text{clean}} \in \mathbb{R}^{500\times 72}$.

Clustering dels pacients. S'agrupen els 72 pacients, cadascun descrit pels seus 500 gens, amb distància de correlació i tres criteris d'enllaç (*single*, *complete* i *average*). Per a cada enllaç es talla l'arbre a $k = 2$ i es calcula l'ARI contra el diagnòstic. El dendrograma i el clustermap es fan amb *average*, que s'ha fixat des del principi (no pas mirant l'ARI) perquè és l'opció estàndard amb distància de correlació en expressió gènica [7]. Amb K-Means es fa servir $k = 2, \ldots, 8$, amb 10 inicialitzacions i llavor fixa; per a cada k es guarden la inèrcia i la silueta mitjana, i per a $k = 2$ l'ARI.

Mòduls de gens i mapes de calor. S'agrupen els 500 gens, cadascun descrit pels seus valors en els 72 pacients, amb distància de correlació i enllaç *average*. Per a $k = 2, \ldots, 10$ es talla l'arbre, es calcula la silueta amb la matriu de distàncies precalculada i es tria el k de silueta màxima, que defineix el mòdul de cada gen. Després es representa X_{clean} en dues versions: sense reordenar (ordre original dels pacients i ordre de selecció dels gens) i reordenada pels dos dendrogrames (clustermap), amb barres de color pel diagnòstic i pel mòdul. Comparar-les mostra què aporta el clustering.

Implementació. El codi és en Python: NumPy i pandas per a les dades, SciPy per al clustering jeràrquic, scikit-learn [10] per a K-Means, la silueta i l'ARI, i Matplotlib i seaborn per als gràfics. Cada pas és una funció amb *docstring*, totes les llavors són fixes i el repositori inclou un `README` i un `requirements.txt`; totes les figures i xifres d'aquest informe s'obtenen d'una única execució del programa.

5.4 Resultats i anàlisi

La Figura 7 mostra l'efecte del retall i del logaritme: la distribució original, molt asimètrica i amb valors negatius, queda acotada entre $\log_2(101) \approx 6{,}66$ i $\log_2(16001) \approx 13{,}97$. Els dos pics extrems corresponen als valors retallats (47,0 % al terra i 0,8 % al sostre), una limitació del retall perquè tots els gens poc expressats queden al mateix valor. El filtre per variància conserva els 500 gens amb variància $\geq 1{,}30$.

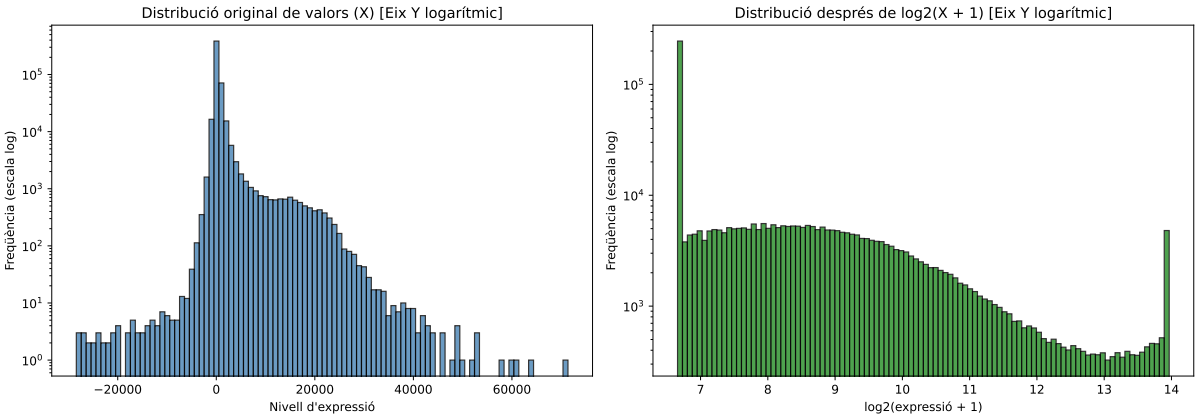


Figura 7: Distribució de les intensitats d'expressió abans (esquerra) i després (dreta) del retall a $[100, 16000]$ i de la transformació $\log_2(x + 1)$. L'eix vertical és logarítmic.

La Taula 5 resumeix els quatre mètodes sobre els 72 pacients i la Figura 8 mostra el dendrograma de l'enllaç *average*.

- **Single** falla: separa un únic pacient i deixa la resta junta (ARI ≈ 0) per l'efecte cadena.
- **Complete** té el millor ARI entre els jeràrquics (0,787), per sobre de 0,7.
- **Average**, l'enllaç triat per al dendrograma i el clustermap, queda just per sota (0,690): l'enllaç condiciona el resultat.

Mètode	Distància	Mides dels 2 grups	ARI
Single	correlació	[71, 1]	−0,013
Complete	correlació	[29, 43]	0,787
Average	correlació	[31, 41]	0,690
K-Means	euclidiana	–	**0,890**

Taula 5: Comparació de mètodes per a $k = 2$ (mides reals: 47 ALL i 25 AML). En negreta, l'ARI màxim (K-Means).

Cap partició té mides 47/25, així que cap recupera exactament el diagnòstic; tanmateix, **K-Means** és la que s'hi acosta més (ARI 0,890).

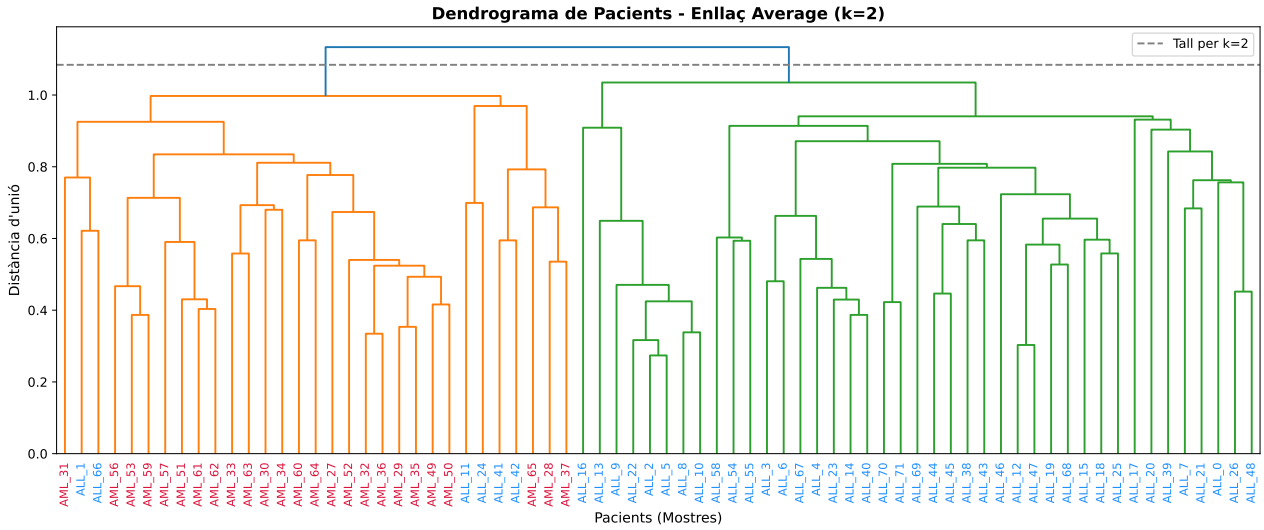


Figura 8: Dendrograma dels 72 pacients (enllaç *average*, distància de correlació). Les etiquetes són blaves per a ALL i vermelles per a AML; la línia grisa discontínua és el tall a $k = 2$.

Pel que fa a K-Means (Figura 9), la inèrcia baixa de manera contínua (de 30.821 a $k = 2$ fins a 22.841 a $k = 8$) i no mostra un colze clar, de manera que el mètode del colze no ajuda a triar k. La silueta, en canvi, és màxima a $k = 2$ (0,1352) i decreix fins a 0,0683 a $k = 8$, la qual cosa coincideix amb els dos diagnòstics reals. Els valors són baixos (inferiors a 0,14): els grups són reconeixibles però no estan gaire separats en un espai de 500 dimensions.

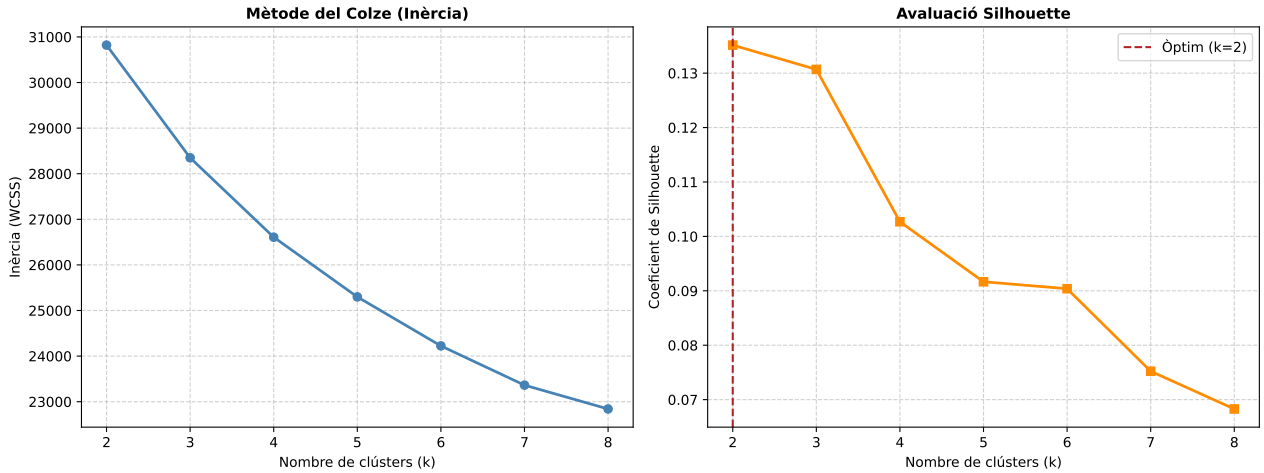


Figura 9: Mètode del colze (esquerra) i coeficient de Silhouette (dreta) per a K-Means.

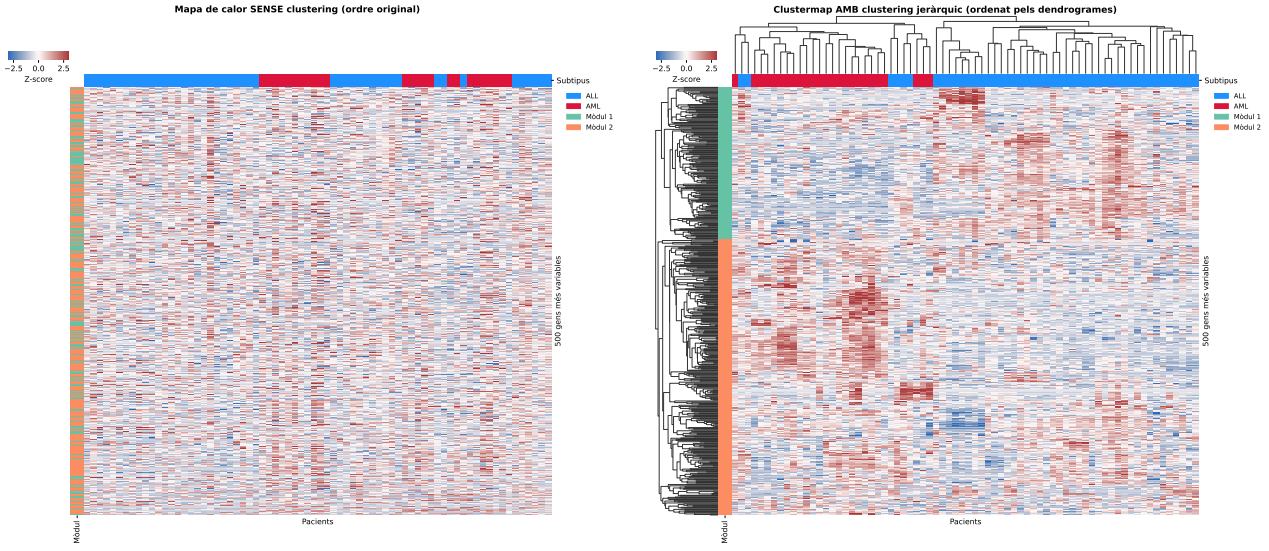
Per als mòduls de gens (Taula 6), la silueta és màxima per a $k = 2$ (0,1812), amb dos mòduls de 178 i

322 gens. Per a k més gran apareixen mòduls molt petits (de 3, 4 o 7 gens) i la silueta empitjora, tot i que té un màxim local a $k = 10$ (0,1394), que donaria una partició més fina. Es manté $k = 2$ per criteri de silueta; és una divisió gruixuda que informa de dos grans patrons d'expressió i no de mòduls funcionals concrets.

k	Silueta	Mides dels mòduls
2	**0,1812**	178, 322
3	0,1302	178, 7, 315
4	0,1188	178, 7, 278, 37
5	0,1088	178, 7, 278, 15, 22
6	0,1125	49, 129, 7, 278, 15, 22
7	0,0929	49, 129, 3, 4, 278, 15, 22
8	0,1297	49, 129, 3, 4, 159, 119, 15, 22
9	0,1276	49, 129, 3, 4, 159, 119, 15, 4, 18
10	0,1394	49, 129, 3, 4, 159, 27, 92, 15, 4, 18

Taula 6: Silueta i mida dels mòduls de gens per a cada k (enllaç *average*, distància de correlació).

Per entendre què aporta el clustering, la Figura 10a mostra la matriu X_{clean} sense reordenar: columnes en l'ordre original dels pacients i files en l'ordre de selecció dels gens (de variància creixent). Cada cel · la és el z-score d'un gen en un pacient (vermell: expressió per sobre de la mitjana d'aquell gen; blau: per sota). En aquest ordre no hi ha cap criteri que agrupi gens o pacients semblants i els patrons no es poden llegir. La Figura 10b mostra la mateixa matriu reordenada pels dos dendrogrames: els valors no canvien, només l'ordre, i els gens i pacients amb perfils semblants queden adjacents. Així apareixen blocs de color i les barres de diagnòstic i de mòdul s'agrupen en trams contigus. Es veu que els pacients AML estan gairebé ben separats, ja que la majoria formen un bloc contigu, i a simple vista el mòdul 2 sembla més expressat en els AML i el mòdul 1 en molts dels ALL.



(a) Sense clustering.

(b) Amb clustering jeràrquic.

Figura 10: Z-score dels 500 gens més variables en els 72 pacients, sense reordenar (a) i reordenat pels dendrogrames de gens i pacients, enllaç *average* (b). Barres de color: diagnòstic (ALL blau, AML vermell) i mòdul de gens.

5.5 Limitacions

Primer, l'alta dimensionalitat: amb 500 característiques i només 72 pacients, les distàncies tendeixen a concentrar-se i la silueta és baixa (0,14–0,18) encara que els grups coincideixin amb el diagnòstic. Segon, la sensibilitat al preprocessament: el retall, la selecció de 500 gens i el logaritme afecten els resultats i no s'ha fet una anàlisi de sensibilitat sistemàtica (per exemple, amb 100, 500 i 1.000 gens).

Tercer, el tall a $k = 2$: l'ARI es calcula amb el nombre de diagnòstics coneguts i, per als jeràrquics, aquest valor és una decisió prèvia (la silueta de K-Means també indica $k = 2$). Quart, per als mòduls de gens s'ha usat directament *average*, sense comparar-lo amb la resta d'enllaços.

A més, es treballa amb un sol conjunt de dades petit, amb mostres de diversos hospitals, de manera que el clustering podria no distingir els efectes de laboratori del senyal biològic. Els mòduls no s'han anotat biològicament i, per tant, no es pot afirmar que tinguin significat funcional. Tampoc s'ha fet cap detecció d'anomalies, així que el pacient aïllat per *single* no s'ha analitzat; queda pendent per a una possible continuació del projecte.

5.6 Conclusions

Els resultats permeten respondre els quatre objectius. Sense usar etiquetes, K-Means (ARI 0,890) i l'enllaç *complete* (0,787) superen l'ARI de 0,7 plantejat i recuperen en gran part la distinció ALL/AML, mentre que *average* (0,690) hi queda just per sota; cap partició, però, reprodueix exactament els grups 47/25. La comparació de mètodes mostra que l'enllaç condiciona el resultat: *single* falla per l'efecte cadena i *complete* i *average* donen ARI diferents, de manera que convé validar amb una mètrica externa quan n'hi ha. Pel que fa als gens, la silueta en tria dos mòduls (178 i 322), una divisió gruixuda i de silueta baixa que no equival a mòduls funcionals, de manera que aquest objectiu s'assoleix només en part. Finalment, el clustermap integra gens i pacients amb els diagnòstics i els mòduls, i mostra què aporta el clustering respecte del mapa de calor brut. En conjunt, l'aprenentatge no supervisat, amb un preprocessament acurat i una distància adequada, recupera bona part de l'estructura biològica sense usar les etiquetes. Com a línies de millora es plantegen una anàlisi de sensibilitat al nombre de gens i al preprocessament, la detecció d'anomalies (Isolation Forest), la comparació amb DBSCAN i models de mescla gaussiana, l'anotació biològica dels mòduls i la repetició de l'estudi amb dades de seqüenciació d'ARN, que requereixen un preprocessament diferent.

5.7 Fórmules complementàries

K-Means. Minimitza la inèrcia $J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2$ alternant dos passos fins que res canvia: assignar cada punt al centre més proper i recalcular cada centre com la mitjana dels seus punts.

Adjusted Rand Index. Sigui n_{ij} el nombre de pacients que són al grup i de la partició trobada i a la classe j del diagnòstic, amb sumes per files a_i i per columnes b_j, i n el total. Llavors

$$\mathrm{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \Big[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\Big] \Big/ \binom{n}{2}}{\frac{1}{2}\Big[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\Big] - \Big[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\Big] \Big/ \binom{n}{2}}.$$

6 Problema d'Uriel: classificació de materials elèctrics amb DBSCAN

Material a `problemas/5_Materials_Uriel/` (`materiales_uriel.py`, `descargar_materiales.py`, informe propi `informe_materiales_uriel.tex` i presentació `presentacion_materiales_uriel.pptx`).

Objectiu. Descobrir i classificar de forma totalment no supervisada les tres famílies quàntiques de l'estat sòlid (conductors metàl · lics, semiconductors tecnològics i aïllants dielèctrics) a partir de l'espai bidimensional $(\log_{10} \rho_{300\mathrm{K}}, \alpha)$, sense utilitzar etiquetes prèvies.

Fonament físic. La resistivitat elèctrica $\rho_{300\mathrm{K}}$ abasta més de 30 ordres de magnitud (des de la plata a $1.6 \times 10^{-8}\ \Omega \cdot \mathrm{m}$ fins al tefló a $> 10^{22}\ \Omega \cdot \mathrm{m}$), exigint compressió logarítmica per evitar que el 99 % dels materials col · lapsin a l'origen. El coeficient tèrmic $\alpha = \frac{1}{\rho_0} \frac{d\rho}{dT}$ delata l'estructura quàntica de bandes:

- **Conductors (metalls):** $\alpha > 0$ ($+0.003$ a $+0.006$ K^{-1}) per dispersió amb fonons al mar d'electrons lliures. Cúmul ultradens a $\log_{10}\rho \approx -8$.
- **Semiconductors:** $\alpha < 0$ (-0.01 a -0.07 K^{-1}) per activació tèrmica exponencial segons Boltzmann ($n \propto e^{-E_g/2k_BT}$). Cúmul intermedi delimitat per la frontera física $\alpha = 0$.
- **Aïllants (dielèctrics):** $\alpha \approx 0$ amb resistivitats extremes ($\log_{10}\rho > 11$) per tenir un bandgap colossal ($E_g > 5$ eV). Cúmul molt dispers al llarg de 12 ordres.

Per què DBSCAN i no K-Means. K-Means assumeix variàncies homogènies i grups esferoidals; en tenir metalls hiperdensos davant d'aïllants dispersos, K-Means fractura els aïllants o absorbeix erròniament els semiconductors, a més d'exigir fixar K. DBSCAN connecta per densitat local arbitrària, tolera densitats desiguals i aïlla anomalies sense prefixar k.

Dades. `datos/materiales_electricos.csv`: 57 materials reals recopilats de taules empíriques estàndard (CRC Handbook, Kittel), desglossats en 35 conductors, 12 semiconductors i 10 aïllants. La columna `clase_real` només s'utilitza per avaluació a posteriori.

Mètode.

1. **Estandardització manual Z-Score:** $Z = (X - \mu)/\sigma$ vectorialment amb NumPy. Imprescindible perquè la distància euclidiana ponderi per igual el rang de $\log_{10}\rho$ (≈ 31 unitats) i el d'α (≈ 0.08 unitats).
2. **DBSCAN amb scikit-learn:** Radi $\varepsilon = 0.65$ (determinat analíticament mitjançant la corba de distàncies k-NN i optimització de silueta) i $v_{\min} = 3$ (regla $k \geq d + 1$ en 2 dimensions).
3. **Avaluació:** Matriu de contingència enfront de les classes reals i càlcul de Silhouette Score ($S = 0.7858$).

Resultats.

- **Clústers descoberts:** 3 clústers purs; 0 punts de soroll.
- **Puresa experimental:** 100 % de puresa en conductors (35/35) i 100 % en aïllants (10/10). En semiconductors, 10 de 12 s'agrupen al Clúster 2; 2 semimetalls amb resistència cuasimetàl · lica a 300 K es van assignar al Clúster 0.
- **Silhouette Score:** $S = 0.7858$, que certifica una separació espacial forta i contundent sense supervisió.

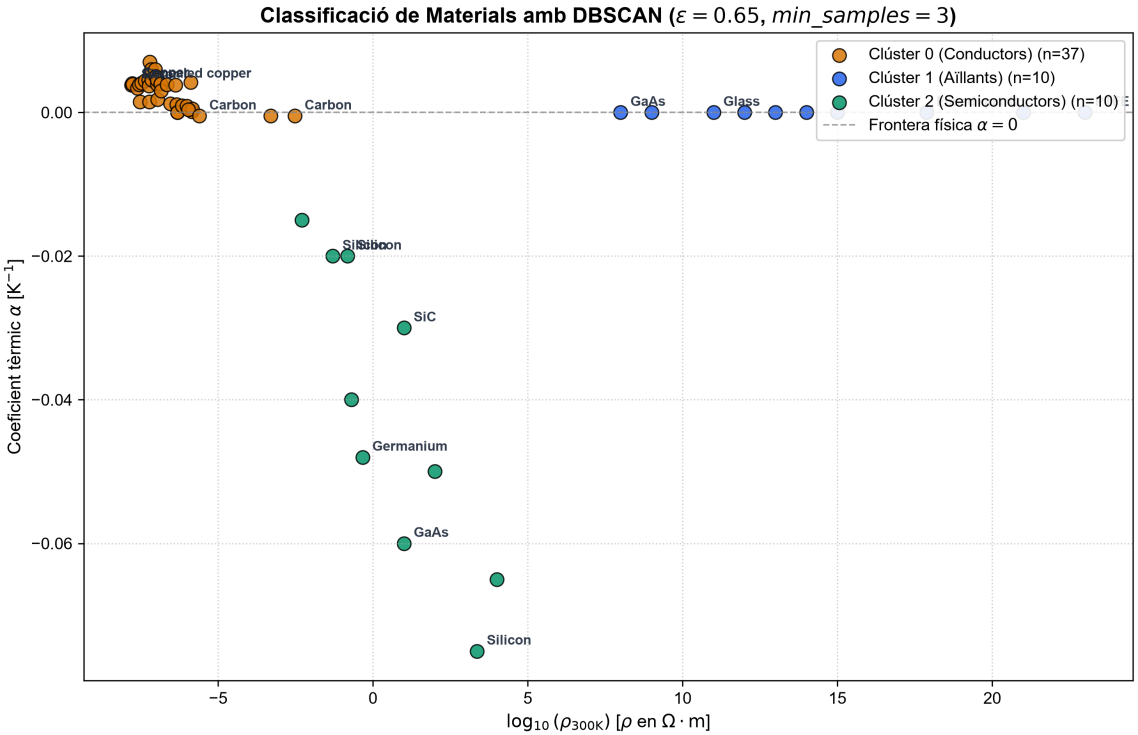


Figura 11: Problema d'Uriel: classificació de 57 materials en el pla $(\log_{10}\rho, \alpha)$ amb DBSCAN $(\varepsilon = 0.65, v_{\min} = 3)$. En ambre metalls, en blau aïllants i en verd semiconductors.

Reproduir. `python problemas/5_Materials_Uriel/descargar_materiales.py` i després `python problemas/5_Materials_Uriel/materiales_uriel.py`.

6.1 Codi font

```python
"""problemas/5_Materials_Uriel/materiales_uriel.py

Clasificación no supervisada de materiales eléctricos (Conductores, Semiconductores y Aislantes)
mediante DBSCAN de scikit-learn (Density-Based Spatial Clustering of Applications with Noise).

Parámetros fijados:
- Epsilon (radio de vecindad): eps = 0.65
- Min samples (mínimo de muestras por núcleo): min_samples = 3

Autor: Uriel
Asignatura: CPIA · Aprendizaje No Supervisado
"""
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# ============================================================================
# 1. Cargar el dataset descargado en datos/
# ============================================================================
DATOS_DIR = Path(__file__).resolve().parents[2] / "datos"
CSV_PATH = DATOS_DIR / "materiales_electricos.csv"

if not CSV_PATH.exists():
    raise FileNotFoundError(f"No se encontró el archivo de datos en: {CSV_PATH}")

df = pd.read_csv(CSV_PATH)
```

```
print(f"Dataset cargado con éxito ({len(df)} materiales).")

# ============================================================================
# 2. Selección de características y estandarización manual (Z-Score)
# ============================================================================
# x1: log10(resistividad a 300 K) -> comprime más de 30 órdenes de magnitud
# x2: coeficiente térmico alpha -> firma cuántica del material
features = ["log_rho_300K", "alpha_coef"]
X_raw = df[features].values

# Estandarización manual: Z = (X - μ) / σ
# Garantiza media = 0 y desviación típica = 1 en cada dimensión
media = np.mean(X_raw, axis=0)
desv = np.std(X_raw, axis=0)
X_scaled = (X_raw - media) / desv

# ============================================================================
# 3. Aplicación de DBSCAN (scikit-learn)
# ============================================================================
EPS = 0.65
MIN_SAMPLES = 3

dbscan = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES)
df["cluster"] = dbscan.fit_predict(X_scaled)

# ============================================================================
# 4. Cálculo del Silhouette Score (scikit-learn)
# ============================================================================
mask_validos = df["cluster"] != -1
if len(set(df["cluster"][mask_validos])) > 1:
    score_sil = silhouette_score(X_scaled[mask_validos], df["cluster"][mask_validos])
else:
    score_sil = None

# ============================================================================
# 5. Métricas y análisis de resultados
# ============================================================================
clusters_encontrados = sorted(set(df["cluster"]) - {-1})
n_clusters = len(clusters_encontrados)
n_ruido = int(np.sum(df["cluster"] == -1))

print("\n" + "=" * 65)
print(f"RESULTADOS DE DBSCAN (eps={EPS}, min_samples={MIN_SAMPLES})")
print("=" * 65)
print(f"Clusters descubiertos por densidad : {n_clusters}")
print(f"Puntos clasificados como ruido (-1): {n_ruido} de {len(df)}")
if score_sil is not None:
    print(f"Silhouette Score (núcleos densos) : {score_sil:.4f}")

print("\nMatriz de contingencia (Clase real vs. Cluster DBSCAN):")
print(pd.crosstab(df["clase_real"], df["cluster"], margins=True))

if n_ruido > 0:
    ruido_df = df[df["cluster"] == -1][["material", "log_rho_300K", "alpha_coef", "clase_real"]]
    print("\nMateriales clasificados como ruido/anomalía (cluster = -1):")
    for _, r in ruido_df.iterrows():
        print(f" - {r['material']:<30} | log10(ρ)={r['log_rho_300K']:>6.2f} | α={r['alpha_coef
    ']:>8.5f} | Tipo: {r['clase_real']}")

# ============================================================================
# 6. Visualización científica en 2D con Matplotlib
# ============================================================================
plt.rcParams['font.sans-serif'] = 'Arial'
plt.rcParams['font.family'] = 'sans-serif'

```

```
plt.figure(figsize=(10, 6.5))

paleta_colores = {
    0: ("#d97706", "Clúster 0 (Conductors)"),
    1: ("#2563eb", "Clúster 1 (Aïllants)"),
    2: ("#059669", "Clúster 2 (Semiconductors)"),
}

for cl in clusters_encontrados:
    sub = df[df["cluster"] == cl]
    color, nombre_cl = paleta_colores.get(cl, ("#6b7280", f"Clúster {cl}"))
    plt.scatter(
        sub["log_rho_300K"],
        sub["alpha_coef"],
        c=color,
        s=70,
        alpha=0.85,
        edgecolors="black",
        linewidth=0.8,
        label=f"{nombre_cl} (n={len(sub)})"
    )

# Puntos de ruido si existieran
sub_ruido = df[df["cluster"] == -1]
if len(sub_ruido) > 0:
    plt.scatter(
        sub_ruido["log_rho_300K"],
        sub_ruido["alpha_coef"],
        c="#dc2626",
        marker="x",
        s=80,
        linewidth=2,
        label=f"Soroll / Anomalies (n={len(sub_ruido)})",
        zorder=5
    )

# Línea de referencia física para α = 0
plt.axhline(0, color="gray", linestyle="--", linewidth=1, alpha=0.7, label=r"Frontera física $\
    alpha = 0$")

# Etiquetas de texto para materiales clave representativos
materiales_destacados = ["copper", "silver", "silicon", "germanium", "gaas", "glass", "ptfe", "
    quartz", "graphite"]
for _, r in df.iterrows():
    mat = r["material"]
    if any(k in mat.lower() for k in materiales_destacados):
        plt.annotate(
            mat.split("(")[0].strip(),
            (r["log_rho_300K"], r["alpha_coef"]),
            xytext=(6, 4),
            textcoords="offset points",
            fontsize=8.5,
            fontweight="semibold",
            color="#1e293b",
            alpha=0.9
        )

plt.title(f"Classificació de Materials amb DBSCAN ($\\varepsilon = {EPS}$, $min\\_samples = {
    MIN_SAMPLES}$)", fontsize=13, fontweight="bold")
plt.xlabel(r"$\log_{10}(\rho_{300\mathrm{K}})$ [$\rho$ en $\Omega\cdot\mathrm{m}$]", fontsize=11)
plt.ylabel(r"Coeficient tèrmic $\alpha$ [$\mathrm{K}^{-1}$]", fontsize=11)
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend(loc="upper right", framealpha=0.9)
plt.tight_layout()

```

```python
# Guardar figura en la carpeta del problema
FIGURA_PATH = Path(__file__).resolve().parent / "clusters_materiales_dbscan.png"
plt.savefig(FIGURA_PATH, dpi=300)
print(f"\nGráfico guardado en: {FIGURA_PATH}")

try:
    plt.show()
except Exception:
    pass
```

Codi 9: `problemas/5_Materials_Uriel/materiales_uriel.py`

7 Conclusions de l'equip

[PENDENT: comparativa dels quatre problemes, quin algorisme ha funcionat millor i per què.]

Annex

Annex: codi complet del problema d'Arnau

Els tres programes del problema de les explosions nuclears, tal com es van executar. Cada secció de l'informe cita només les línies que s'hi expliquen.

```python
"""
Descarga el catalogo del USGS (todos los eventos M>=4.5 de 1951 a 2025, de cualquier tipo)
y lo guarda en datos/sismos_usgs_M45_1951_2025.csv.
"""
from pathlib import Path
import pandas as pd

DATOS = Path(__file__).resolve().parent.parent.parent / 'datos'
DESTINO = DATOS / 'sismos_usgs_M45_1951_2025.csv'
URL = 'https://earthquake.usgs.gov/fdsnws/event/1/query?format=csv&orderby=time-asc&minmagnitude=4.5'


def descargar_year(year):
    # La API limita cada consulta a 20 000 eventos, asi que se pide un año cada vez
    consulta = f'{URL}&starttime={year}-01-01&endtime={year + 1}-01-01'

    for intento in range(4):
        try:
            return pd.read_csv(consulta)
        except Exception:
            print(f'  {year}: reintento')

    raise RuntimeError(f'No se pudo descargar {year}')


if __name__ == '__main__':
    if DESTINO.exists():
        print(f'{DESTINO.name}: ya esta')
    else:
        DATOS.mkdir(exist_ok=True)

        years = []
        for year in range(1951, 2026):
            eventos = descargar_year(year)
            years.append(eventos)
            print(f'{year}: {len(eventos)} eventos')

        df = pd.concat(years, ignore_index=True)
        df = df.rename(columns={'type': 'tipo'})
        df.to_csv(DESTINO, index=False)

        explosiones = (df['tipo'] == 'nuclear explosion').sum()
        print(f'{DESTINO.name}: {len(df)} eventos, {explosiones} explosiones')
```

Codi 10: `problemas/2_Nuclear/nukedatos.py`: descàrrega del catàleg.

```python
from pathlib import Path
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from sklearn.cluster import DBSCAN

# --- Catalogo y parametros ---
# Catalogo USGS 1951-2025, M>=4.5, todos los eventos (se descarga con nukedatos.py)
CARPETA = Path(__file__).resolve().parent
ARCHIVO = CARPETA.parent.parent / 'datos' / 'sismos_usgs_M45_1951_2025.csv'

# Parametros fijos
#   eps_km: radio de DBSCAN. vmin: sismos minimos por cluster.
```

```
#   dmin, dmax: una explosion es casi superficial, asi que un cluster es sospechoso
#   si la mediana de su profundidad esta entre 0 y 2.5 km.
parametros = dict(eps_km=25, vmin=1, dmin=0, dmax=2.5)

# --- Datos ---
df = pd.read_csv(ARCHIVO)
df = df.dropna(subset=['latitude', 'longitude', 'depth', 'mag'])
df = df[df['mag'] >= 4.5]
df = df.reset_index(drop=True)


# --- Agrupar los sismos cercanos con DBSCAN ---
def agrupar(df, eps_km, vmin):
    radio_tierra_km = 6371
    coordenadas = np.radians(df[['latitude', 'longitude']])
    eps_rad = eps_km / radio_tierra_km

    dbscan = DBSCAN(eps=eps_rad, min_samples=vmin, metric='haversine')
    cluster = dbscan.fit_predict(coordenadas)
    return pd.Series(cluster, index=df.index)


# --- Marcar los sismos sospechosos de ser explosiones ---
def nukeDBSCAN(df, eps_km, vmin, dmin, dmax):
    cluster = agrupar(df, eps_km, vmin)

    # Profundidad mediana de cada cluster (el ruido, -1, no cuenta)
    sin_ruido = cluster != -1
    mediana = df['depth'].groupby(cluster[sin_ruido]).median()

    # Sospechoso: cluster casi en la superficie, con la mediana entre dmin y dmax
    es_superficial = (mediana >= dmin) & (mediana < dmax)
    clusters_sospechosos = mediana[es_superficial].index

    return cluster.isin(clusters_sospechosos)


# --- Falsos positivos y falsos negativos ---
def errores(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'

    # Falso positivo: detectado, pero no es una explosion
    falsos_positivos = (detectado & ~es_explosion).sum()
    # Falso negativo: es una explosion, pero no se detecta
    falsos_negativos = (~detectado & es_explosion).sum()
    return falsos_positivos, falsos_negativos


if __name__ == '__main__':
    # --- Run con los parametros fijos ---
    detectado = nukeDBSCAN(df, **parametros)
    fp, fn = errores(df, detectado)
    print(f'{parametros}: FP={fp}, FN={fn}')

    # --- Grupos de sismos para el globo ---
    es_explosion = df['tipo'] == 'nuclear explosion'
    terremotos_muestra = df[~es_explosion].sample(4000, random_state=0)
    explosiones = df[es_explosion]
    sospechosos = df[detectado]
    aciertos = df[detectado & es_explosion]
    falsos_positivos = df[detectado & ~es_explosion]
    falsos_negativos = df[~detectado & es_explosion]

    # Cada vista del globo es una lista de (sismos, nombre, color)
    vistas = {
```

```
        'Todos los eventos': [(terremotos_muestra, 'Terremotos (muestra)', 'lightgray'),
                              (explosiones, 'Explosiones nucleares', 'red')],
        'Sospechosos': [(sospechosos, 'Sospechosos', 'purple')],
        'Reales': [(explosiones, 'Explosiones nucleares', 'red')],
        'Aciertos y errores': [(aciertos, 'Acierto', 'green'),
                               (falsos_positivos, 'Falso positivo', 'orange'),
                               (falsos_negativos, 'Falso negativo', 'gray')],
    }

    # --- Globo (globo.html) ---
    fig = go.Figure()
    for vista, capas in vistas.items():
        for sismos, nombre, color in capas:
            fig.add_trace(go.Scattergeo(
                lat=sismos['latitude'],
                lon=sismos['longitude'],
                mode='markers',
                marker=dict(color=color, size=5),
                name=f'{nombre} ({len(sismos)})',
                hoverinfo='skip',
                meta=vista,
                visible=(vista == 'Todos los eventos'),
            ))

    # Un boton por vista: muestra las capas de esa vista y oculta las demas
    botones = []
    for vista in vistas:
        visibles = [capa.meta == vista for capa in fig.data]
        botones.append(dict(label=vista, method='update', args=[{'visible': visibles}]))

    fig.update_layout(updatemenus=[dict(buttons=botones, x=0.01, y=0.99)],
                      legend=dict(x=0.01, y=0.1),
                      margin=dict(l=0, r=0, t=0, b=0))
    fig.update_geos(projection_type='natural earth',
                    showland=True, landcolor='whitesmoke', showcountries=True)

    # Plotly va incluido en el HTML: funciona sin internet
    fig.write_html(CARPETA / 'globo.html', include_plotlyjs=True)
    fig.show()
```

Codi 11: `problemas/2_Nuclear/nukeDBSCAN_arnau.py`: detecció i globus interactiu.

```
"""
Validacion de nukeDBSCAN_arnau.py: silueta y distancias k para mirar eps,
estabilidad con el Adjusted Rand Index (ARI) y mapa de aciertos y errores.
"""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.neighbors import NearestNeighbors

from nukeDBSCAN_arnau import parametros, df, agrupar, nukeDBSCAN, errores

CARPETA = Path(__file__).resolve().parent
RADIO_TIERRA_KM = 6371
EPS_PROBADOS = [5, 10, 15, 20, 25, 30, 40, 50, 75, 100]


# --- Silueta de unos clusters (sobre una muestra de 2000 sismos) ---
def silueta(df, cluster):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    try:
        return silhouette_score(coordenadas, cluster, metric='haversine',
                                sample_size=2000, random_state=42)
```

```
    except ValueError:
        # La muestra no tiene al menos 2 clusters
        return np.nan


# --- Para cada eps: numero de clusters, silueta, FP y FN ---
def evaluar_eps(df, vmin, dmin, dmax):
    print('\n--- ELECCION DE EPS ---')
    print(f"{'eps (km)':<10}{'clusters':<10}{'silueta':<10}{'FP':<8}{'FN':<8}")
    print('-' * 46)

    n_clusters = []
    siluetas = []
    for eps in EPS_PROBADOS:
        cluster = agrupar(df, eps, vmin)
        detectado = nukeDBSCAN(df, eps, vmin, dmin, dmax)
        fp, fn = errores(df, detectado)

        n_clusters.append(cluster.nunique())
        siluetas.append(silueta(df, cluster))
        print(f'{eps:<10}{n_clusters[-1]:<10}{siluetas[-1]:<10.3f}{fp:<8}{fn:<8}')

    return n_clusters, siluetas


# --- Distancia de cada sismo a su 4.o vecino, de mayor a menor (el "codo" de DBSCAN) ---
def distancias_k(df, k=4):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    vecinos = NearestNeighbors(n_neighbors=k, metric='haversine').fit(coordenadas)
    distancias, _ = vecinos.kneighbors(coordenadas)

    distancia_k = distancias[:, -1] * RADIO_TIERRA_KM
    return np.sort(distancia_k)[::-1]


# --- Estabilidad: ARI entre los clusters del catalogo y los de una muestra del 50% ---
def evaluar_estabilidad(df, eps_km, vmin):
    print(f'\n--- ESTABILIDAD (eps={eps_km} km, muestra del 50%) ---')
    cluster_completo = agrupar(df, eps_km, vmin)

    for semilla in [1, 2, 3, 4, 5]:
        muestra = df.sample(frac=0.5, random_state=semilla)
        cluster_muestra = agrupar(muestra, eps_km, vmin)

        ari = adjusted_rand_score(cluster_completo[muestra.index], cluster_muestra)
        print(f'Semilla {semilla}: clusters={cluster_muestra.nunique()}, ARI={ari:.4f}')


# --- Figura 1: clusters, silueta y distancias k ---
def figura_eps(n_clusters, siluetas, distancia_k):
    fig, ejes = plt.subplots(1, 3, figsize=(14, 3.8))

    ejes[0].plot(EPS_PROBADOS, n_clusters, 'o-')
    ejes[0].set(xlabel='eps (km)', ylabel='Numero de clusters', title='Clusters segun eps')

    ejes[1].plot(EPS_PROBADOS, siluetas, 'o-', color='green')
    ejes[1].set(xlabel='eps (km)', ylabel='Silueta', title='Silueta segun eps')

    ejes[2].plot(distancia_k)
    ejes[2].set(xlabel='Sismos ordenados', ylabel='Distancia al 4.o vecino (km)',
                yscale='log', title='Distancias k (codo)')

    for eje in ejes:
        eje.grid(True, linestyle='--', alpha=0.5)

```

```
    fig.tight_layout()
    fig.savefig(CARPETA / 'nukeval_eps.png', dpi=200)


# --- Figura 2: mapa de aciertos y errores ---
def figura_mapa(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'
    otros = df[~detectado & ~es_explosion]

    fig, eje = plt.subplots(figsize=(10, 5))
    eje.scatter(otros['longitude'], otros['latitude'], s=1, c='lightgray')

    grupos = [(detectado & es_explosion, 'green', 'Acierto'),
              (detectado & ~es_explosion, 'orange', 'Falso positivo'),
              (~detectado & es_explosion, 'black', 'Falso negativo')]
    for mascara, color, nombre in grupos:
        sismos = df[mascara]
        eje.scatter(sismos['longitude'], sismos['latitude'], s=8, c=color,
                    label=f'{nombre} ({len(sismos)})')

    eje.set(xlabel='Longitud (°)', ylabel='Latitud (°)',
            title='Aciertos y errores con los parametros fijos')
    eje.legend(loc='lower left', markerscale=2)
    eje.grid(True, linestyle='--', alpha=0.5)

    fig.tight_layout()
    fig.savefig(CARPETA / 'nukeval_mapa.png', dpi=200)


if __name__ == '__main__':
    print(f'Catalogo: {len(df)} eventos, parametros {parametros}')

    n_clusters, siluetas = evaluar_eps(df, parametros['vmin'], parametros['dmin'], parametros['
    dmax'])
    evaluar_estabilidad(df, parametros['eps_km'], parametros['vmin'])

    detectado = nukeDBSCAN(df, **parametros)
    fp, fn = errores(df, detectado)
    print(f'\nRun con los parametros fijos: FP={fp}, FN={fn}')

    figura_eps(n_clusters, siluetas, distancias_k(df))
    figura_mapa(df, detectado)
    print(f'Figuras guardadas en: {CARPETA}')
```

Codi 12: `problemas/2_Nuclear/nukeval.py`: validació de ε i estabilitat.

Referències

[1] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD-96)*, 226–231.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] Sinnott, R. W. (1984). Virtues of the haversine. *Sky & Telescope*, 68(2), 159.

[4] U.S. Geological Survey. *ANSS Comprehensive Earthquake Catalog (ComCat)*. https://earthquake.usgs.gov/data/comcat/

[5] U.S. Department of Energy, Nevada Operations Office (2015). *United States Nuclear Tests, July 1945 through September 1992*, DOE/NV-209 Rev. 16. https://www.osti.gov/biblio/1351809

[6] Golub, T. R. et al. (1999). Molecular classification of cancer: class discovery and class prediction by gene expression monitoring. *Science*, 286(5439), 531–537.

[7] Eisen, M. B., Spellman, P. T., Brown, P. O., Botstein, D. (1998). Cluster analysis and display of genome-wide expression patterns. *PNAS*, 95(25), 14863–14868.

[8] Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

[9] Hubert, L., Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2, 193–218.

[10] Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[11] Yang, X., North, R., Romney, C., & Richards, P. G. (2003). Worldwide nuclear explosions. A: W. H. K. Lee et al. (eds.), *International Handbook of Earthquake and Engineering Seismology*, Part B, cap. 84. Academic Press.

[12] International Atomic Energy Agency (1998). *The Radiological Situation at the Atolls of Mururoa and Fangataufa: Main Report*. IAEA, Viena.

[13] Tian, D., Yao, J., & Wen, L. (2018). Collapse and earthquake swarm after North Korea's 3 September 2017 nuclear test. *Geophysical Research Letters*, 45(9), 3976–3983. https://doi.org/10.1029/2018GL077649

[14] Slemmons, D. B. (1957). Geological effects of the Dixie Valley–Fairview Peak, Nevada, earthquakes of December 16, 1954. *Bulletin of the Seismological Society of America*, 47(4), 353–375.

[15] Ford, S. R., Dreger, D. S., & Walter, W. R. (2009). Identifying isotropic events using a regional moment tensor inversion. *Journal of Geophysical Research: Solid Earth*, 114, B01306. https://doi.org/10.1029/2008JB005743

[16] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.

[17] Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

[18] Arthur, D., & Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035.

[19] Schubert, E., Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (2017). DBSCAN revisited, revisited: why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems*, 42(3), 19. https://doi.org/10.1145/3068335

[20] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proc. 8th IEEE International Conference on Data Mining*, 413–422. https://doi.org/10.1109/ICDM.2008.17